ITA0615 – MACHINE LEARNING
CO4 AT2

By:
Harshith G
192525394

1: RETAIL PRODUCT RECOMMENDATION USING K-NEAREST NEIGHBOUR (KNN)

Problem understanding:

The retailer has a history of which customers bought (or rated) which products. The goal is to suggest products a customer is likely to buy next. KNN is suitable because it is simple, needs no training phase and works directly on past purchase records (user-based collaborative filtering).

Proposed solution:

- Data: build a customer × product matrix where each cell is 1/0 (bought or not) or the rating/quantity. Optional features: category preference, average basket value, purchase frequency and recency.
- Pre-processing: remove duplicates, fill missing values with 0 (not purchased), and normalise numeric features (min-max) so that no feature dominates the distance.

How similarity is measured:

- Each customer is a vector of purchases. Similarity between customers A and B is measured with cosine similarity: $\text{sim}(A, B) = (A \cdot B) / (\|A\| \|B\|)$, which suits sparse purchase data. For numeric profile features the Euclidean distance $d = \sqrt{\Sigma(a_i - b_i)^2}$ is used, with similarity $= 1 / (1 + d)$. Jaccard similarity can be used for pure bought / not-bought data.
- A smaller distance (or larger cosine value) means that two customers have similar buying behaviour.

How recommendations are generated:

- Step 1: find the K most similar customers to the target customer (K is chosen by cross-validation, typically 5 – 20).
- Step 2: collect the products bought by these neighbours but not yet bought by the target customer.
- Step 3: score each product by the similarity-weighted vote: $\text{score}(p) = \Sigma\ \text{sim}(\text{target, neighbour}) \times \text{rating}(\text{neighbour, p})$.
- Step 4: rank the products by score and show the top-N as recommendations.

How the system adapts to new customer data:

- KNN is a lazy learner, so a new purchase is simply added to the stored data and is used immediately in the next similarity search; no retraining is required.
- New customers (cold start) are handled by recommending popular products until enough purchases are available, or by matching on profile data such as age group and location. Old records can be down-weighted or removed so that recent behaviour counts more.

Conclusion:

KNN gives an easy-to-explain, continuously updated recommender ("customers like you also bought …"), which increases cross-selling and customer satisfaction. For very large catalogues, approximate nearest-neighbour search (KD-tree, LSH) keeps it fast.

2: MACHINE OUTPUT PREDICTION USING LOCALLY WEIGHTED REGRESSION (LWR)

Problem understanding:

Machine output depends on sensor readings (temperature, vibration, pressure, speed, etc.) whose relationship with the output changes with time and operating conditions. A single global model such as ordinary linear regression cannot follow this changing, non-linear behaviour. LWR is therefore proposed.

Proposed solution:

- Collect time-stamped sensor readings x(i) and the corresponding machine output y(i), and store them as the training set (LWR is a memory-based method).
- For every new reading x_q, a separate linear model is fitted using only the data close to x_q, and that model gives the prediction.
- Weights are given by a Gaussian kernel: $w(i) = \exp(-\|x(i) - x_q\|^2 / 2\tau^2)$, where τ is the bandwidth. The weighted least squares solution is $\theta = (X^T W X)^{-1} X^T W y$ and the prediction is $\hat{y} = \theta^T x_q$.
- Time can be included as a feature, or the kernel can give extra weight to recent readings, so the model follows drift in machine behaviour.

How local models improve prediction accuracy:

- Nearby readings receive high weights and distant readings receive almost zero weight, so each prediction uses a model that is valid in that operating region (e.g. warm-up, normal load, high load) instead of one average line for all regions.
- This approximates any smooth non-linear curve with many small linear pieces and reduces bias, which gives lower error than global linear regression.
- The bandwidth τ controls the bias–variance trade-off: small τ follows the data closely (risk of over-fitting), large τ behaves like global regression (under-fitting). τ is selected by cross-validation.

Handling noisy or incomplete data:

- Noise: the weighted averaging over many neighbouring points smooths random sensor noise. Robust LWR (re-weighting with residuals, as in LOWESS) reduces the effect of outliers, and a moving-average filter can be applied to raw signals.
- Missing readings: records with missing values are imputed (previous value / mean of neighbours) or the missing feature is excluded when computing distances; faulty sensors are flagged with range checks.
- Regularisation: a small ridge term λI is added, $\theta = (X^T W X + \lambda I)^{-1} X^T W y$, so that the matrix stays invertible when few neighbours are available.

Conclusion:

LWR gives accurate, adaptive predictions of machine output under changing conditions and can be used for planning, quality control and predictive maintenance. Its limitation is the cost of fitting a model for each query, which is reduced by using only the nearest neighbours (KD-tree) and by a sliding window of recent data.

3: PATIENT RISK CLASSIFICATION USING K-NEAREST NEIGHBOUR (KNN)

Problem understanding:

The company wants to place each patient in a risk category (low, medium or high) using historical medical data such as age, BMI, blood pressure, glucose, cholesterol, heart rate and past diagnoses. KNN is chosen because it is simple, makes no assumption about the data distribution and gives results that doctors can interpret ("patients similar to this one were high risk").

Proposed solution:

- Pre-processing: handle missing values (median imputation), remove outliers, encode categorical data (e.g. smoker: yes/no) and standardise all features (z-score) because KNN depends on distances.
- A new patient is compared with all stored patients using the Euclidean (or weighted Euclidean) distance, the K nearest patients are found and the majority risk class among them is assigned. For medical safety, the vote can be weighted by $1/d^2$ and the class probability (e.g. 80% high risk) is reported.
- K is selected by k-fold cross-validation (an odd K avoids ties).

Handling large datasets:

- Brute-force search costs $O(n \cdot d)$ per query, so efficient structures are used: KD-tree or ball-tree for low dimensions, and approximate nearest-neighbour methods (LSH, HNSW) for large, high-dimensional data.
- Data reduction: condensed nearest neighbour or clustering (keeping only representative patients) reduces storage, and the search can be run in parallel or in batches.

Feature selection:

- Irrelevant features distort the distance, so important features are chosen by correlation analysis, mutual information, chi-square tests or wrapper methods (forward/backward selection with cross-validated accuracy), together with medical expert advice.
- PCA can reduce dimensionality and remove correlated features; feature weights can be learned so that clinically important features (e.g. blood glucose) contribute more to the distance.

Improving accuracy in real-world scenarios:

- Handle class imbalance (few high-risk patients) using SMOTE or class-weighted voting, and judge the model with recall, precision, F1-score and ROC-AUC, not accuracy alone, because missing a high-risk patient is costly.

- Tune K and the distance measure (Euclidean, Manhattan, Mahalanobis), use distance-weighted voting, and periodically add new patient records so the model stays current.
- Validate with cross-validation and a separate hospital data set; keep patient data secure and anonymised (privacy rules such as HIPAA/GDPR).

Conclusion:

A properly scaled, feature-selected and tuned KNN classifier can support early identification of high-risk patients, with efficient indexing used to keep it fast on large records. It should assist, not replace, the doctor's decision.

4: SIMILAR PRODUCT DETECTION FOR DYNAMIC PRICING USING CASE-BASED LEARNING (CBL / CBR)

Problem understanding:

The e-commerce platform has to set or change the price of a product according to the prices and performance of similar products. Case-Based Reasoning solves a new problem by retrieving similar past cases and reusing their solutions, which fits this task well: a new or changing product is matched with similar earlier products and their successful prices are reused.

Case representation:

Each case = problem description + solution + outcome. Problem: category, brand, specifications (e.g. RAM, size), cost, rating, number of reviews, demand, stock level, season, competitor price. Solution: the price (or discount) that was applied. Outcome: units sold, revenue and profit margin obtained.

How similarity metrics are defined:

- Numeric features (cost, rating, size): normalised distance, e.g. Euclidean distance on min-max scaled values.
- Categorical features (category, brand): exact match = 1, otherwise 0 (or a category-tree similarity so that 'laptop' is closer to 'tablet' than to 'shirt').
- Text features (title, description): TF-IDF or embedding vectors compared with cosine similarity.
- Overall similarity is a weighted sum: $Sim(Q, C) = \Sigma\, w_i \cdot sim_i(Q_i, C_i) / \Sigma\, w_i$, where the weights w_i express feature importance (e.g. category and specifications get high weight). The weights are tuned using past pricing results.

How past cases are reused (CBR cycle):

- Retrieve: find the K most similar products (cases) to the new product.
- Reuse: use their successful prices – for example a similarity-weighted average price – as the proposed price.
- Revise: adjust the proposed price for current conditions (competitor price, stock level, demand, minimum margin rules) and, if needed, test it with A/B testing.
- Retain: store the new product, its final price and the observed sales outcome as a new case, so that future pricing improves automatically.

How the system scales with increasing product data:

- Indexing: use category-based indexing and nearest-neighbour structures (KD-tree, LSH, HNSW) so that only a small subset of cases is compared.
- Case-base maintenance: remove duplicate, outdated or low-quality cases and keep representative ones (clustering) to control the size; periodically recompute feature weights.
- Use distributed storage and batch / parallel processing for retrieval, and cache the results for popular products.

Conclusion:

Case-Based Learning gives transparent and adaptive dynamic pricing: the price is justified by similar past products, and the system learns from every new sale. This increases revenue and competitiveness while keeping prices explainable.

5: DELIVERY-TIME FORECASTING USING RADIAL BASIS FUNCTION (RBF) NETWORKS

Problem understanding:

Delivery time depends non-linearly on location (distance, route type), traffic patterns (time of day, day of week, congestion level), weather and the vehicle load. A linear model cannot capture such interactions, so an RBF network is proposed.

RBF network design:

- Input layer: distance, origin/destination coordinates, hour of day, day of week, traffic index, weather, number of stops (normalised).
- Hidden layer: Gaussian units $\varphi_j(x) = \exp(-\|x - c_j\|^2 / 2\sigma_j^2)$. The centres c_j are found by K-Means clustering of past deliveries (each centre represents a typical situation such as 'peak-hour city traffic'), and σ_j is set from the spread of the cluster (e.g. $\sigma = d_max / \sqrt{(2m)}$).
- Output layer: a linear combination $\hat{y} = \Sigma\, w_j\, \varphi_j(x) + b$, which is the predicted delivery time. The weights are obtained quickly by linear least squares ($w = (\Phi^T\Phi)^{-1}\, \Phi^T\, y$) or by gradient descent, so training is fast.

How the model captures non-linear relationships:

- Every Gaussian unit responds strongly only to inputs close to its centre. The weighted sum of many such local bumps can approximate any smooth non-linear function (universal approximation), e.g. a sharp rise in delivery time during rush hours in the city centre but not on highways.
- Interactions between variables (location × time × traffic) are captured automatically because the distance is calculated in the combined input space.

How reliability is improved under varying conditions:

- Selecting the number of centres and the widths by cross-validation avoids under-fitting and over-fitting; regularisation (ridge term on the weights) improves stability.
- The model is retrained periodically (or updated online) with the latest deliveries, so that new road works, seasons and festival traffic are learned. Live traffic and weather feeds are used as inputs at prediction time.
- Outliers (accidents, breakdowns) are filtered or handled with robust loss, and prediction intervals (e.g. from residual error or an ensemble of RBF networks) are given so that the customer sees a delivery window such as '2 – 3 hours' instead of a single value.
- Performance is monitored with MAE, RMSE and on-time-delivery percentage; if the error increases, the model is recalibrated.

Conclusion:

The RBF network provides fast, accurate and adaptive delivery-time forecasts for varying traffic and location conditions. It improves route planning, resource allocation and customer satisfaction. Alternatively, Locally Weighted Regression can be used, but the RBF network is faster at prediction time because it stores only the centres, not the complete data set.